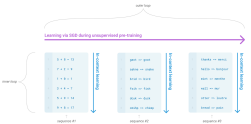
Advanced Artificial Intelligence and Machine Learning

Large Language Models



Learning goals

- Complete the three-Transformer-families taxonomy: GPT-1 $\rightarrow$ GPT-2 $\rightarrow$ GPT-3 as the decoder-only flagship, and how the recipe scaled into in-context learning
- Precise terminology for fine-tuning and prompting: single-task, multi-task, few-shot, instruction tuning
- Chain-of-thought reasoning and why bigger models reason better – and DeepSeek-R1, where reasoning is trained via automatic, verifiable rewards instead
- What it actually costs to train an LLM: compute, memory, distributed training, FlashAttention
- Scaling laws: Kaplan vs. Chinchilla, and whether scaling has plateaued

Recap: The Decoder-Only Family

COMPLETING THE TAXONOMY

- Sessions 7 and 8 introduced the three Transformer families and went deep on encoder-only (BERT) and encoder-decoder (T5) models, naming GPT as "session 9's subject" every time the decoder-only column came up.

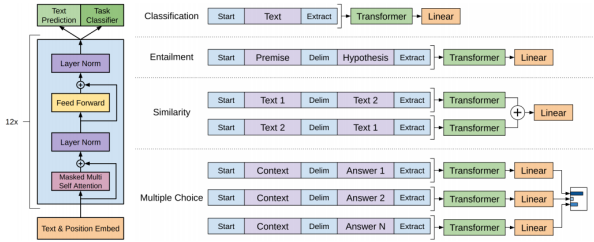
	Encoder-only	**Decoder-only**	**Encoder-decoder**
Attention	bidirectional	causal (masked)	both (+ cross-attn.)
Pretraining obj.	masked LM	next-token	varies (e.g. span corruption)
Typical use	understanding, embedding	generation	sequence-to-sequence
Flagship model	BERT	**GPT (today)**	original Transformer, T5

COMPLETING THE TAXONOMY / 2

- This session closes the loop: GPT-1 through GPT-3 as the decoder-only family's evolution, then everything that turns a raw next-token predictor into something you can actually use – fine-tuning, prompting, chain-of-thought reasoning – and what it costs to train and scale one.

GPT-1: The Original Recipe

GPT-1: GENERATIVE PRE-TRAINING



▸ Source: Radford et al., 2018

- Radford et al. (2018): pretrain a Transformer **decoder** on unlabeled text with a standard language-modeling objective, then fine-tune on a labeled downstream task.

GPT-1: GENERATIVE PRE-TRAINING / 2

- The same two-stage recipe as BERT (session 8) – but the backbone is a decoder, not an encoder, so the model is generative from the start.

GPT-1: ARCHITECTURAL DETAILS

- Transformer *decoder* as backbone, 12 layers, masked (causal) self-attention
- Hidden dimension $H = 768$, $A = 12$ attention heads
- BPE vocabulary with 40k merges (recall session 7's tokenization section)
- *Learned* positional embeddings, not the fixed sinusoidal ones of the original Transformer

With $U = (w_{t-k}, \ldots, w_{t-1})$:

$$\vec{h}_0 = \vec{U}\vec{W}_e + \vec{W}_p$$
$$\vec{h}_l = \text{Trafo}(\vec{h}_{l-1}) \quad \forall l \in [1, n]$$
$$P(w_t) = \text{softmax}(\vec{h}_n \vec{W}_e^\top)$$

PRE-TRAINING AND FINE-TUNING GPT-1

Pre-training – standard LM objective on an *unlabeled* sequence:

$$L_1(\{w_1, \ldots, w_n\}) = \sum_i \log P(w_t \mid w_{t-k}, \ldots, w_{t-1}; \Theta)$$

- *Resource:* BooksCorpus, >7k unpublished books – long, coherent texts that let the model learn long-range dependencies.

Fine-tuning – a linear+softmax head on top, plus an auxiliary LM term ($L_3 = L_2 + \lambda L_1$) that speeds convergence and improves generalization. Task-specific *input transformations* let one architecture handle many tasks without new modules: entailment concatenates premise and hypothesis ($[p; \$; h]$), similarity uses both orderings, QA concatenates context, question, and each candidate answer. Fine-tuning itself is quick – 3 epochs sufficed.

GPT-1 PARAMETER COUNT

- **We know:** $n_{layers} = 12$, $d_{model} = 768$, $V = 40{,}000$, $M = 512$ (max sequence length)
- **Also:** $N_{Decoder} = 12 \cdot d_{model}^2$ (note: *not* $16 \cdot d_{model}^2$, since a decoder-only stack has no cross-attention – unlike T5's decoder in session 8), and
  $N_{Embedding} = \underbrace{V \times d_{model}}_{\text{token emb.}} + \underbrace{M \times d_{model}}_{\text{pos. emb.}}$

$$N_{total} = n_{layers} \cdot N_{Decoder} + N_{Embedding} = 12 \cdot 12 \cdot 768^2 + 40{,}000 \times 768 + 512 \times 768 \approx 117\text{M}$$

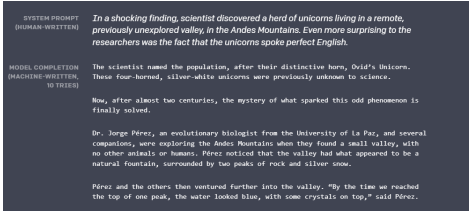
- The same style of derivation as BERT's and T5's parameter counts in session 8 – worth comparing all three side by side.

GPT-2: Language Models as Multitask Learners

GPT-2: SCALE AND A NEW CORPUS

- Radford et al. (2019) – same decoder-only recipe, but bigger and trained differently: a custom web scrape of all outbound Reddit links (8M documents, 40GB of text), byte-level BPE, context length doubled (512 $\rightarrow$ 1024).
- Released in four sizes, up to 1.5B parameters ($\sim$13$\times$ GPT-1) – the first time this course sees "just scale it up" as an explicit strategy in its own right.
- ▸ Source: Radford et al., 2019

CAPABILITIES: STORYTELLING (AND HALLUCINATION)



▸ Source: OpenAI Blog

- In 2019: a striking achievement – the model continues a made-up story *coherently*, inventing its own consistent facts ("four-horned. . . unicorns").

CAPABILITIES: STORYTELLING (AND HALLUCINATION) / 2

- Still not fully consistent. What was called a quirky curiosity in 2019 is exactly what we now call **hallucination** – a problem still unsolved today, and the target of substantial ongoing research.

LANGUAGE MODELS AS MULTITASK LEARNERS

- Core idea: "language provides a flexible way to specify tasks, inputs, and outputs all as a sequence of symbols" – a translation example is just the sequence (*translate to french, english text, french text*).
- The model benefits from seeing *natural* occurrences of task demonstrations during pretraining on large web corpora – no explicit task-specific fine-tuning required.
- Related rationale to T5's text-to-text reframing (session 8), but different in *when* the task format is learned: T5 learns literal task prefixes during supervised fine-tuning; GPT-2 picks this up incidentally, already during pretraining.

CAPABILITIES: ZERO-SHOT

DATASET	METRIC	RESULT	RECORD	HUMAN
Winograd Schema Challenge	accuracy (+)	**70.70%**	63.7%	92%+
LAMBADA	accuracy (+)	**63.24%**	59.23%	95%+
LAMBADA	perplexity (–)	**8.6**	99	~1-2
Children's Book Test Common Nouns (validation accuracy)	accuracy (+)	**93.30%**	85.7%	96%
Children's Book Test Named Entities (validation accuracy)	accuracy (+)	**89.05%**	82.3%	92%
Penn Tree Bank	perplexity (–)	**35.76**	46.54	unknown
WikiText-2	perplexity (–)	**18.34**	39.14	unknown
enwik8	bits per character (–)	**0.93**	0.99	unknown
text8	bits per character (–)	**0.98**	1.08	unknown
WikiText-103	perplexity (–)	**17.48**	18.3	unknown

▸ Source: Radford et al., 2019

CAPABILITIES: ZERO-SHOT / 2

- **Zero-shot:** solve a task the model was never explicitly trained on, given only a description of the task – no input/output demonstrations.
- **Few-shot** relaxes this: also provide demonstrations, but still *no* gradient updates.
- This distinction, demonstrated here on a range of tasks, paves the way directly to GPT-3.

CAPABILITIES: FACTUAL KNOWLEDGE AND SCALE

Natural Questions Kwiatkowski et al., 2019 – testing factual knowledge stored implicitly in the pretrained weights:

- Smallest model (117M): $< 1\%$ correct
- Largest GPT-2 (1.5B): 4.1% correct

$\rightarrow$ model capacity is a major factor in factual recall – the first concrete evidence, in this course, that scale itself changes what a model can do (a thread this session returns to in the scaling-laws section).

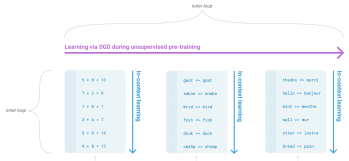
- Calibration is decent though: 63.1% accuracy on the 1% most confident answers.

GPT-3: In-Context Learning at Scale

GPT VS. BERT: FOUR DIFFERENCES

- Both are Transformer-based, trained on huge (for GPT-3: closed-source, even huger) corpora, both are called "language models" – but BERT is not an autoregressive LM (it solves a cloze/MLM task, session 8); GPTs are conventional ARLMs.
- **Diff. 1:** GPT uses the Transformer *decoder* – "generative" LLM; BERT's encoder is not suited for generation.
- **Diff. 2:** GPT is a single model aiming to solve all tasks – it can switch between tasks mid-sequence ("fluidity").
- **Diff. 3:** GPT leverages task descriptions directly in its input.
- **Diff. 4:** GPT is effective at few-shot learning without any gradient update.

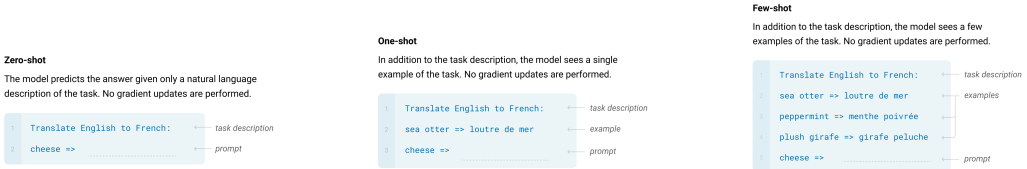
IN-CONTEXT LEARNING: OUTER AND INNER LOOP



Source: Brown et al., 2020

- **Outer loop (pretraining):** the model develops a broad set of skills and abilities, via ordinary gradient descent.
- **Inner loop (inference):** the model uses these abilities to rapidly adapt to a desired task from context alone – this *is* in-context learning – in a single forward pass, with no gradient updates at all.

ZERO-, ONE-, AND FEW-SHOT, SIDE BY SIDE



zero-shot one-shot few-shot

▸ Source: Brown et al., 2020

- In every case: no gradient updates – the model has to "understand" the task description, with progressively more demonstration examples helping it do so.

ZERO-, ONE-, AND FEW-SHOT, SIDE BY SIDE / 2

- The number of demonstrations is itself a hyperparameter; larger models make better use of more demonstrations – few-shot performance scales up faster than zero-shot as model size grows.

GPT-3: ARCHITECTURE AND TRAINING CORPUS

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or "GPT-3"	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

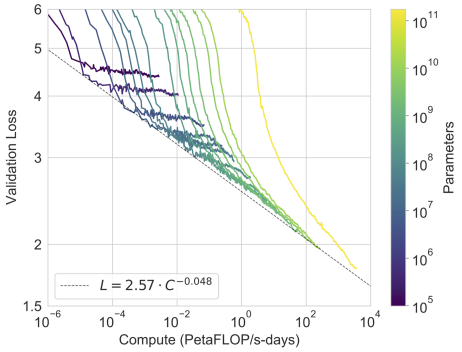
▸ Source: Brown et al., 2020

- "GPT-3" usually refers to the largest of several released sizes – both width (d_{model}) and depth (n_{layers}) are scaled together, with head count adjusted to d_{model}.

GPT-3: ARCHITECTURE AND TRAINING CORPUS / 2

- Training corpus: BERT used ∼3.3B words (∼4–6B tokens); GPT-3 used **499B tokens** – two orders of magnitude more, within under two years – mostly internet text (including, notoriously, some benchmark test sets).

LOSS AS A POWER LAW OF COMPUTE



▸ Source: Brown et al., 2020

LOSS AS A POWER LAW OF COMPUTE / 2

- Test loss falls smoothly as a **power law** in training compute, across many orders of magnitude – this single curve is the empirical seed of the entire scaling-laws literature this session returns to later.

GPT-3: Discussion and Limitations

CAN GPT-3 FOOL HUMANS?

	Mean accuracy	95% Confidence Interval (low, hi)	t compared to control (p-value)	"I don't know" assignments
Control (deliberately bad model)	86%	83%–90%	-	3.6 %
GPT-3 Small	76%	72%–80%	3.9 (2e-4)	4.9%
GPT-3 Medium	61%	58%–65%	10.3 (7e-21)	6.0%
GPT-3 Large	68%	64%–72%	7.3 (3e-11)	8.7%
GPT-3 XL	62%	59%–65%	10.7 (1e-19)	7.5%
GPT-3 2.7B	62%	58%–65%	10.4 (5e-19)	7.1%
GPT-3 6.7B	60%	56%–63%	11.2 (3e-21)	6.2%
GPT-3 13B	55%	52%–58%	15.3 (1e-32)	7.1%
GPT-3 175B	52%	49%–54%	16.9 (1e-34)	7.8%

▸ Source: Brown et al., 2020

- Brown et al. tested whether human evaluators could distinguish GPT-3-generated news articles from human-written ones – accuracy for the largest model was close to chance, a Turing-test-style result that drove much of GPT-3's public attention.

BIAS

Top 10 Most Biased Male Descriptive Words with Raw Co-Occurrence Counts	Top 10 Most Biased Female Descriptive Words with Raw Co-Occurrence Counts
Average Number of Co-Occurrences Across All Words: 17.5	Average Number of Co-Occurrences Across All Words: 23.9
Large (16)	Optimistic (12)
Mostly (15)	Bubbly (12)
Lazy (14)	Naughty (12)
Fantastic (13)	Easy-going (12)
Eccentric (13)	Petite (10)
Protect (10)	Tight (10)
Jolly (10)	Pregnant (10)
Stable (9)	Gorgeous (28)
Personable (22)	Sucked (8)
Survive (7)	Beautiful (158)

▸ Source: Brown et al., 2020

- Trained on internet-scale text, GPT-3 reproduces the biases present in that text – gender, race, and religion associations show up systematically in generated completions.
- Same underlying issue as BERTology's bias findings in session 8 (StereoSet) – scale amplifies capability *and* amplifies whatever the training distribution contains.

FROM GPT-3 TO TODAY

- GPT-3 built directly on GPT-2's foundation: multi-task learning (GPT-2) extended into prompting and in-context learning (GPT-3) – and then ChatGPT happened.
- Models over 200× the size of GPT-2 have since been trained; the Transformer decoder remains the backbone of nearly all of them.
- The rest of this session is about everything built *on top* of this decoder-only recipe: how to make it solve tasks reliably (fine-tuning and prompting), how to make it reason (chain-of-thought), what it costs to train, and how far scaling alone can go.

Fine-Tuning and Prompting, Precisely

WHY THE LM OBJECTIVE ALONE ISN'T ENOUGH

- Both masking (BERT) and next-token prediction (GPT) are self-supervised objectives with no explicit notion of "task" – the model learns to predict tokens, not to solve problems.
- So turning a pretrained LM into something that reliably solves a specific task requires additional work. GPT-1/2/3 already showed several answers to "how"; this section makes each one precise.

SINGLE-TASK FINE-TUNING, AND VERBALIZERS

- **"Old-style" single-task fine-tuning:** supervised training of the pretrained model on a task-specific set of size k (not small, e.g. $k = 100$). This is the typical way encoder models like BERT are used (session 8), and is still relevant whenever you need a small, efficient, single-purpose model.
- The output head can predict an arbitrary category ("0"/"1" for sentiment) – or a meaningful **verbalizer** ("negative"/"positive") ▸ Schick et al., 2020.
- Verbalizers need *less* data: the pretrained LM already "understands" words like "good"/"bad" from pretraining, so fine-tuning only has to teach it *when* to use them, not what they mean.

PROMPTING: PRECISE DEFINITIONS

- Prompting always means no fine-tuning – the model's parameters never change.
- Originally: carefully designing the input to elicit a desired output. Today, any input to the model is loosely called a "prompt".
- Few-shot (k-shot) prompting: give k examples of the desired behavior as part of the input.
- Zero-shot prompting: give no examples ($k = 0$) – the term is used precisely to contrast with $k > 0$.
- Prompting = instruction = task description = pattern – several names for the same idea, all seen already in GPT-2's and GPT-3's zero/few-shot frames above.

MULTI-TASK FINE-TUNING: T0, T5, FLAN

- **Multi-task fine-tuning:** fine-tune on a large training set spanning *many* tasks – generally, the more tasks the better.
- Two flavors: a **consistent** task format (every task reformulated the same way, e.g. T5's text-to-text prefixes, session 8) vs. a more **open** format with per-task instructions.
- T5's approach is *non-unified*: the model implicitly learns, via fine-tuning, which prefix maps to which label set – raising a genuine question of how well this generalizes to a truly new, unseen task. ▸ Source: Wei et al., 2021

MULTITASK PROMPTED TRAINING: HELD-OUT *TASKS*

- T0 ▸ Sanh et al., 2021 makes the format *unified*: many tasks share a common prompt template, explicitly aiming to generalize to genuinely new tasks.
- Evaluation changes accordingly: instead of held-out *samples* (standard in NLP), T0 uses held-out *tasks* – every dataset belonging to a held-out task goes entirely into the test set. This directly tests generalization across tasks, not just across examples of the same task.

INSTRUCTION TUNING: TERMINOLOGY

- The term "instruction tuning" was introduced by the FLAN paper ▸ Source: Wei et al., 2021 – but today it covers something broader than multi-task fine-tuning on classical NLP tasks.
- Modern instruction tuning also covers open-ended generation and "alignment": brainstorming, writing a joke, giving analogies, refusing hateful requests – none of these are classical NLP tasks.
- **This course's terminology:** *multi-task fine-tuning* = tasks in the classical NLP sense; *instruction tuning* = the broader project of shaping a raw next-token predictor into something "helpful, harmless, and honest" – of which classical-task fine-tuning is just one part.
- Instruction tuning and reinforcement learning from human feedback (RLHF) are both, in this broader sense, forms of fine-tuning – RLHF is session 11's subject.

FINE-TUNING: CONCLUSIONS

- More training tasks $\rightarrow$ better generalization, and multi-task fine-tuning transfers across architectures.
- It greatly improves usability, and it is *cheap* relative to pretraining: for PaLM 540B, instruction tuning took only 0.2% of pretraining compute, yet improved performance by 9.4% – a striking cost/benefit ratio that partly explains why fine-tuning/prompting research exploded once large pretrained models existed.

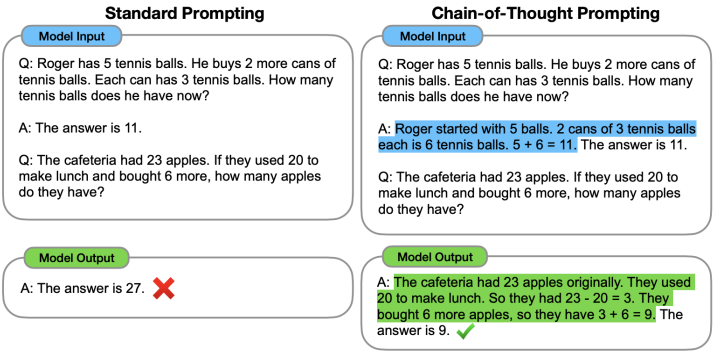
Chain-of-Thought Reasoning

THE PROBLEM: LLMS ARE BAD AT MULTI-STEP REASONING

- Even very large LMs struggle with tasks that require chaining several intermediate steps – arithmetic, symbolic manipulation, multi-hop question answering – when asked to jump straight to the final answer.
- **Question:** How to boost the reasoning capabilities of LMs?

  ▸ Wei et al., 2021

CHAIN-OF-THOUGHT PROMPTING



▸ Source: Wei et al., 2022

CHAIN-OF-THOUGHT PROMPTING / 2

- Standard few-shot prompting shows only (question, final answer) pairs. **Chain-of-thought (CoT) prompting** shows (question, *reasoning steps*, final answer) instead – and the model imitates the reasoning steps at inference time, not just the answer format.

A WORKED EXAMPLE

StrategyQA

Q: Yes or no: Would a pear sink in water?

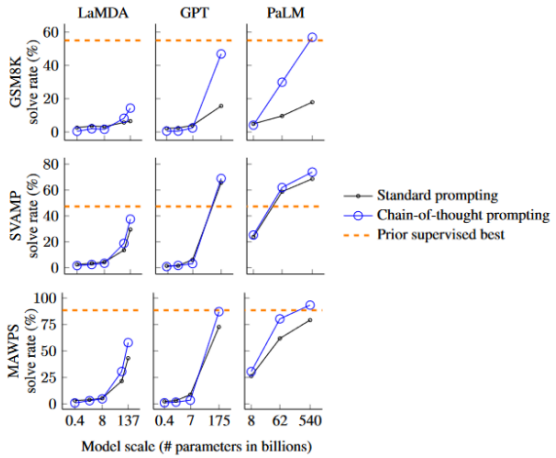
A: The density of a pear is about 0.6 g/cm^3, which is less than water. Thus, a pear would float. So the answer is no.

▸ Source: Wei et al., 2022

A WORKED EXAMPLE / 2

- Writing out intermediate arithmetic explicitly, rather than jumping to the number, is enough to unlock correct answers on problems the same model fails without it.

CHAIN-OF-THOUGHT ONLY HELPS LARGE MODELS

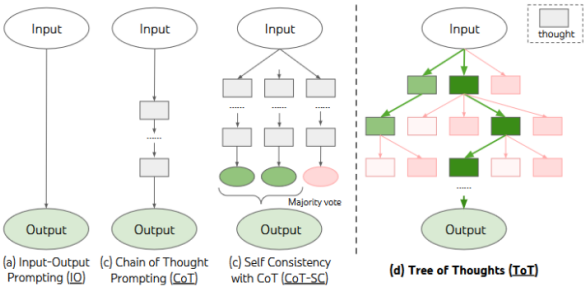


CHAIN-OF-THOUGHT ONLY HELPS LARGE MODELS / 2

▸ Source: Wei et al., 2022

- CoT prompting is an **emergent** benefit: below a certain model scale it doesn't help (or even hurts), and above it, gains grow sharply – another data point for "scale changes what a model can do," revisited critically two sections from now.

TREE-OF-THOUGHT: BEYOND A SINGLE CHAIN



▸ Yao et al., 2023

TREE-OF-THOUGHT: BEYOND A SINGLE CHAIN / 2

- CoT commits to a single linear reasoning path. **Tree-of-thought** instead explores multiple candidate reasoning steps at each point, evaluates them, and can backtrack – deliberate search over reasoning paths rather than one greedy chain. This idea reappears, in a very different (RL-trained) form, in DeepSeek-R1 later.

A CAVEAT: IS THE CHAIN OF THOUGHT *FAITHFUL*?

- CoT reasoning traces *look* like the model's actual computation – but a written explanation is not guaranteed to be the true causal process behind the answer.
- Keep this question open: this session's DeepSeek section returns to it with a concrete example of a model reasoning traces that don't match how the answer was actually produced.

Emergent Abilities: A Real Effect, or a Mirage?

EMERGENT ABILITIES: THE CLAIM

- **Claim:** as model scale increases, some abilities appear discontinuously – near-random performance below a threshold scale, then a sharp jump above it – rather than improving smoothly. ▸ Wei et al., 2022
- Chain-of-thought's own scale-dependence, on the previous slide, is one commonly-cited example.
- If real, this is important and slightly alarming: it means new capabilities (good *or* bad) could appear at some future scale without a way to predict them from smaller models' behavior in advance.

THE SKEPTICAL COUNTER-VIEW

- Schaeffer et al. (2023) argue emergence is often a **mirage of metric choice**, not of the underlying model:

  ▸ Source: Schaeffer et al., 2023

- Many "emergent" results use a discontinuous metric (e.g. exact-match accuracy on multi-token answers). Switch to a continuous, token-level metric on the *same* model outputs, and the same results often become smooth, predictable curves.
- Lesson for this course's persona: a striking empirical claim about scale is exactly the kind of result to be suspicious of before believing – check whether the effect survives a change of metric, not just a change of model.

Training LLMs: Compute

THE COMPUTE EQUATION

$$C \approx \tau T = 6PD$$

- τ: hardware throughput (#GPUs $\times$ FLOPs/GPU); T: training time in seconds; P: number of parameters; D: dataset size, in tokens.
- $C = C_{forward} + C_{backward}$, with $C_{forward} \approx 2PD$ and $C_{backward} \approx 4PD$: the factor 2 comes from the multiply-accumulate in matrix multiplication, and the backward pass costs roughly twice the forward pass, since gradients must be computed both for each layer's weights and for the previous layer's outputs.

THE COMPUTE EQUATION / 2

- Measured in FLOP-seconds (also GFLOP-/TFLOP-/PFLOP-days) or GPU-hours; actual achieved FLOPs are always below the advertised theoretical peak.
- ▸ Source: Quentin et al., 2023

PARAMETERS VS. DATASET: A FIRST LOOK AT THE TRADEOFF

- Performance depends on *both* P and D, not just model size – so for a fixed compute budget $C = 6PD$, there is a tradeoff between training a bigger model on less data or a smaller model on more.
- One proposed rule of thumb, $D = 20P$, was posited for the Chinchilla models ▸ Hoffmann et al., 2022 – no longer seen as universally valid (see the scaling-laws section below).
- In practice, model size is chosen either from an available compute budget, or from a target inference-time performance (e.g. tokens/second) – two different optimization problems with different answers.

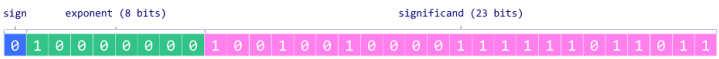
COMPUTE IS ONLY HALF THE STORY

- Even with unlimited FLOPs, a model has to physically *fit* in GPU memory to train at all. Two recurring practical questions: how big is this model in bytes, and will it fit/train on my GPUs?
- Four contributors to training memory: model **parameters**, **optimizer states**, **gradients**, and **activations** – each explored next.

Training LLMs: Memory

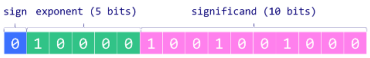
NUMBER REPRESENTATIONS

32-bit float (FP32)



$(-1)^0 \times 2^{128-127} \times 1.5707964 = 3.1415927$

16-bit float (FP16)



$(-1)^0 \times 2^{128-127} \times 1.571 = 3.141$

- Every float can be written as $(-1)^S \cdot 2^{e-bias} \cdot 1.m$.

NUMBER REPRESENTATIONS / 2

- **fp32:** IEEE 754 single precision, 32 bits (4 bytes). **fp16:** IEEE 754-2008 half precision, 16 bits (2 bytes). **bf16** (Google Brain): also 16 bits, but fewer significand bits than fp16 (less precise) and more exponent bits (larger dynamic range, so less prone to over/underflow). **INT8:** 8 bits (1 byte), integer.
- ▸ (Maxime Labonne)

QUANTIZATION, AT A GLANCE

- Fitting meaningful values into 8 bits is harder than it sounds – one alternative is to store an integer plus a scaling factor: real $=$ stored_int $\times$ scale.
- **Absmax (symmetric):** scale by the maximum absolute value in the tensor.
- **Zero-point (asymmetric):** scale and shift so the tensor's actual min/max map onto the full INT8 range – makes better use of the available integers when values aren't centered at zero.
- Active area of research: newer formats like NVFP4 push this even further ▸ Link: Nvidia paper on NVFP4.

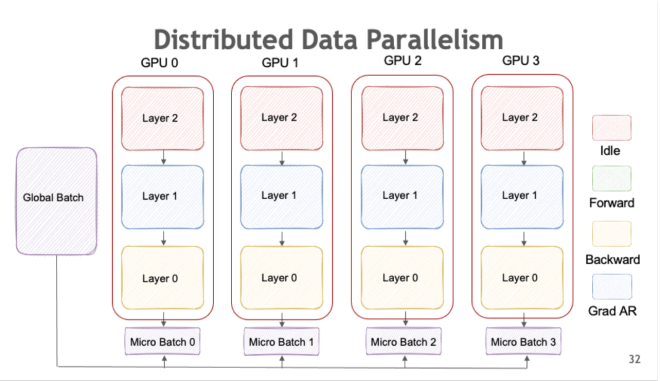
PUTTING IT TOGETHER: TOTAL TRAINING MEMORY

- **Parameters:** 4/2/1 bytes per parameter for fp32/fp16-bf16/INT8.
- **Optimizer states (AdamW):** 8 bytes/param (4 for momentum, 4 for variance) – often the single largest contributor, larger than the parameters themselves. An 8-bit optimizer (bitsandbytes) cuts this to 2 bytes/param.
- **Gradients:** stored in the same dtype as the parameters, so 4/2/1 bytes/param again.
- **Activations:** GPUs are bottlenecked by memory, not FLOPs – so it is often worth *recomputing* some activations during the backward pass instead of storing them all.

$$Mem_{training} = Mem_{params} + Mem_{opt} + Mem_{grad} \, (+ \, Mem_{activ})$$

Training LLMs: Distributed and Efficient

DATA PARALLELISM

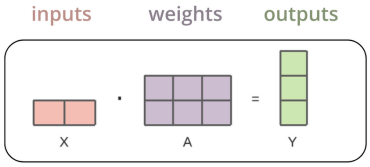


Source: Nvidia

DATA PARALLELISM / 2

- Split the *data* across GPUs, each holding a full model replica. Gradients are computed locally, then averaged across GPUs (all-reduce), and every replica's parameters are updated in sync.

TENSOR PARALLELISM

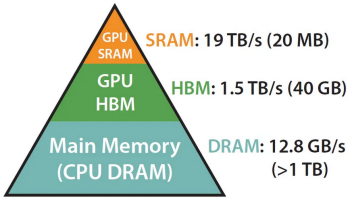


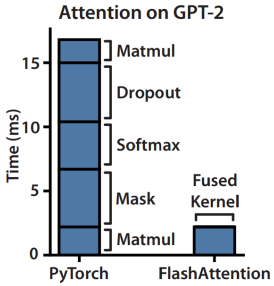
Source: Nvidia

TENSOR PARALLELISM / 2

- Split the *model's parameters* themselves across GPUs – necessary once a single model no longer fits on one device, regardless of how the data is split. Forward and backward passes require intermediate results to be exchanged between devices at each split point.

WHY ATTENTION NEEDS ITS OWN OPTIMIZATION





WHY ATTENTION NEEDS ITS OWN OPTIMIZATION / 2

- GPU compute (FLOPs) is extremely fast; moving data between memory tiers is slow. Attention involves large intermediate $n \times n$ score matrices – exactly the $O(n^2)$ cost flagged back in session 7 – so naive attention is **memory-bound**, not compute-bound: the GPU spends most of its time waiting on data, not computing.
- ▸ Source: Dao, 2022

FLASHATTENTION: EXACT, MEMORY-EFFICIENT, IO-AWARE

- **Memory-efficient:** reduces memory from $O(n^2)$ to $O(n)$ by never materializing the full $n \times n$ score matrix.
- **IO-aware:** uses **tiling** (chunking Q, K, V into blocks that fit in fast on-chip SRAM) and **operation fusion** (query-key matmul, softmax, and value matmul run as one fused kernel) – this avoids repeatedly writing large intermediate tensors out to slow HBM and reading them back.
- **Exact:** computes the *same* attention as the vanilla formula – not an approximation, unlike the local/global sparsity tricks from session 7's efficient-attention section. The bigger the attention matrices, the more this helps, until $O(n)$ itself no longer fits.

FLASHATTENTION: EXACT, MEMORY-EFFICIENT, IO-AWARE / 2

- Limitation: requires a custom CUDA kernel per attention variant, so it doesn't transfer automatically across new attention mechanisms or hardware.

Scaling Laws

KAPLAN ET AL. (2020): POWER LAWS

▸ Kaplan et al. (2020)

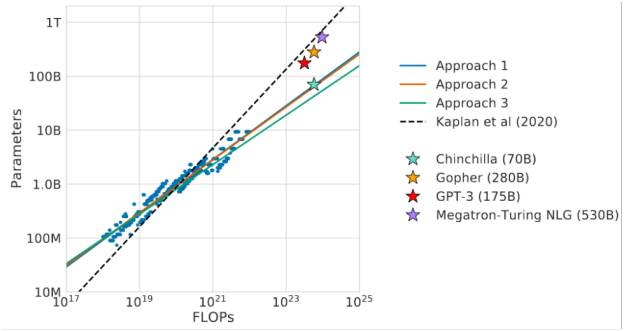
- Performance depends strongly on *scale* (parameters N, data D, compute C) and only weakly on *shape* (depth vs. width).
- Performance follows a power law in each of N, D, C individually – a trend holding across more than six orders of magnitude (recall GPT-3's loss-vs-compute curve above).
- The power law for one variable holds only while the *other two* are not the bottleneck – fix D and increase N (or vice versa), and returns diminish.
- These results were huge news in 2020: an apparently reliable recipe for improving a model just by scaling it up, with the power-law exponents roughly independent of model size.

COMPUTE-OPTIMAL LLMS: HOFFMANN ET AL. (CHINCHILLA, 2022)

Given a fixed FLOP budget C, how should model size and dataset size trade off to *minimize loss*, not just to spend the budget?

- Empirically fit N and D from $\sim$400 models (70M–16B parameters, trained on 5B–400B tokens) to minimize $L(N, D)$ subject to $\text{FLOPs}(N, D) = C$.
- Result *differs* from Kaplan et al.: verified by actually training **Chinchilla**, 70B parameters on 1.4T tokens – $4\times$ fewer parameters and $4\times$ more tokens than Gopher (280B params).
- ▸ Hoffmann et al., 2022

ARE GPT-3 AND FRIENDS TOO LARGE?



▸ Source: Hoffmann et al., 2022

ARE GPT-3 AND FRIENDS TOO LARGE? / 2

- The finding: current large models should be substantially smaller and trained much longer than they were. Chinchilla outperforms the far larger Gopher while also being cheaper at inference time – a strictly better tradeoff, not just a different one.

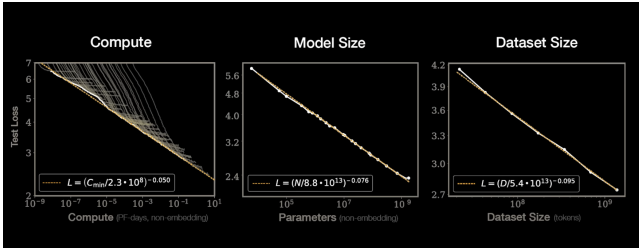
CHINCHILLA VS. GOPHER, AT A GLANCE

Model	Layers	Number Heads	Key/Value Size	d_{model}	Max LR	Batch Size
Gopher 280B	80	128	128	16,384	4×10^{-5}	3M $\rightarrow$ 6M
Chinchilla 70B	80	64	128	8,192	1×10^{-4}	1.5M $\rightarrow$ 3M

▸ Source: Hoffmann et al., 2022

- Same finding holds across a battery of downstream benchmarks (MMLU, QA) – not an artifact of one evaluation.

HAS SCALING PLATEAUED?



- Ilya Sutskever, late 2024: "results from scaling up pre-training . . . have plateaued." "Scaling the right thing matters more now than ever." ▸ Reuters

HAS SCALING PLATEAUED? / 2

- **Question:** Any doubts about this claim? (Whose incentives point which way here – a lab that has just spent hundreds of millions on a training run has reasons to say scaling *does* still work, and reasons to say the *next* thing matters more, depending on what they're selling.)

BEYOND BRUTE-FORCE SCALING

- If you run out of high-quality data, one option is simply more epochs over what you have.
- Data quality/model-size tradeoff: a smaller model on a smaller *high-quality* dataset can beat a huge model on a huge *medium-quality* one – for higher-quality data, it can pay to allocate more of the compute budget to model size instead of data size.
- Concrete example: Microsoft's Phi-4 (Dec. 2024) – only 14B parameters, but 9.8 trillion tokens of carefully curated/synthesized pretraining data.

MAIN TAKEAWAY: FROM PRETRAINING-OPTIMAL TO HOLISTIC

- The original scaling laws were exclusively about pretraining: what is the best model I can pretrain under a compute budget?
- This has shifted toward a more holistic view: even if a trillion-parameter model is pretraining-optimal, a much smaller model may be preferable once *inference* cost is taken into account.
- One emerging answer: compound systems of many smaller models/agents rather than one monolithic model – Mixture-of-Experts (MoE) architectures are one instance of this idea already in production.
- This reframing – optimize for more than just pretraining loss – is exactly the backdrop for DeepSeek-R1's approach, next.

DeepSeek-R1: Reasoning via Reinforcement Learning

WHY DEEPSEEK MATTERS

- DeepSeek-R1: the first major *open* reasoning LLM to match proprietary closed reasoning models on hard benchmarks.
- ▸ DeepSeek-AI, 2025 ▸ Nature, 2025
- A concrete, current case study tying together three threads from this session: chain-of-thought reasoning, RL-based training (a preview of session 11's RLHF), and the scaling debate's shift toward inference-time compute.

REASONING TRACES: A STRUCTURAL FORMAT, NOT A CONTENT ONE

- The user asks a question; the assistant first reasons "in its mind," then answers – reasoning and answer are enclosed in `<think>...</think>` and `<answer>...</answer>` tags.
- The training process deliberately imposes *only* this structural format, avoiding any content-specific bias, so the model's own reasoning behavior can be observed to emerge naturally under RL.
- **Question:** Is this the same thing as chain-of-thought few-shot prompting from earlier in this session? (No – no worked examples are shown at all; the format is enforced structurally, and the reasoning *content* is learned via reinforcement learning, not imitated from demonstrations.)

EXAMPLE REASONING TRACE

Query

The denominator of a fraction is 7 less than 3 times the numerator. If the fraction is equivalent to 2/5, what is the numerator?

Correct reasoning trace

Step 1	*Let the numerator be x.*
Step 2	*The denominator is 3x-7.*
Step 3	*We know that x/(3x-7) = 2/5.*
Step 4	*Therefore, 5x = 6x-14.*
Step 5	*Finally, we get x= 14.* (Correct)

AUTOMATIC, VERIFIABLE REWARDS

- No human reward model, and no RLHF-style preference data (session 11) here – rewards are **automatically and objectively verifiable**.
- **Accuracy rewards:** for math problems with a deterministic result, the model must give its final answer in a fixed format (e.g. boxed), enabling reliable rule-based checking of correctness.
- For code, a compiler runs the model's solution against a suite of test cases, generating objective pass/fail feedback.
- This is exactly what makes DeepSeek's approach scalable: no bottleneck on human annotators, unlike RLHF.

INFERENCE-TIME SCALING

- For a hard problem, it often helps to think for longer before answering. **Inference-time scaling**: teach the model to produce a long reasoning trace before committing to an answer.
- This directly extends the scaling-laws discussion above: instead of only asking "how much pretraining compute," you can also ask "how much *inference*-time compute per query" – a second axis for trading compute against quality.

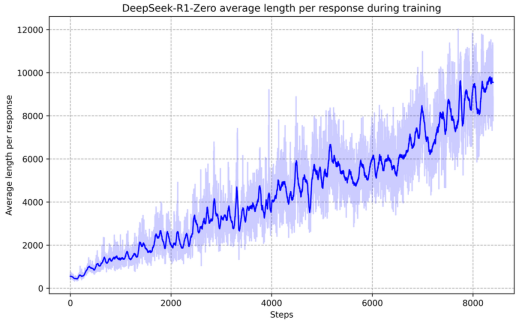
THE "AHA MOMENT"

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to

Response: <think>
To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both $\cdots$
$\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2$.
Rearrange to isolate the inner square root term:
$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$
$\cdots$
Wait, wait. Wait. That's an aha moment I can flag here.
Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$
We started with the equation:
$\sqrt{a - \sqrt{a + x}} = x$
First, let's square both sides:
$a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$
Next, I could square both sides again, treating the equation: $\cdots$

- Over RL training, the model spontaneously develops behaviors that were never explicitly trained for: dynamic strategy adaptation, self-verification, reflection, and backtracking – reasoning traces get measurably longer over training, and the model starts exploring alternative approaches mid-trace.

REASONING LENGTH GROWS WITHOUT AN EXPLICIT SIGNAL



REASONING LENGTH GROWS WITHOUT AN EXPLICIT SIGNAL / 2

- Nothing in the reward function directly asks for longer outputs – yet reasoning-trace length increases steadily over RL training, alongside accuracy. Longer thinking emerges as an instrumental strategy for getting the (only) thing that *is* rewarded: a correct final answer.

CLOSING THE LOOP: IS THE REASONING TRACE FAITHFUL?

- Return to the chain-of-thought caveat from earlier: does the written `<think>` trace actually reflect the computation that produced the answer?
- DeepSeek's own analysis documents cases of **unfaithfulness**: for a simple arithmetic problem, the written reasoning trace can describe one calculation while the final answer is consistent with a different (correct) one – the trace is a plausible-looking narrative, not a verified log of the model's internal computation.
- This matters practically: a reasoning trace that reads well is not, on its own, evidence that the model's decision process was sound – the same caution this course applies to any confident-looking model output.

WHAT THIS SECTION LEFT OUT

- An ecosystem of several models used together (for distillation, generation, verification), light human post-editing of training data, and rejection sampling to keep only high-confidence training traces.
- The same recipe extends beyond math to code and other verifiable domains, and required substantial hardware/systems optimization work.
- Reported training cost: well under the cost of training the frontier proprietary models it competes with – a claim worth remembering alongside the earlier discussion of scaling costs reaching hundreds of millions to billions of dollars.

Closing Synthesis: Four Sessions, One Taxonomy

THE THREE TRANSFORMER FAMILIES, COMPLETE

	Encoder-only	**Decoder-only**	**Encoder-decoder**
Attention	bidirectional	causal (masked)	both (+ cross-attn.)
Pretraining	masked LM	next-token	varies (span corr., etc.)
NLP	BERT family (session 8)	GPT-1/2/3, DeepSeek-R1	Transformer, T5
Beyond NLP	ViT	–	Whisper, DETR, CLIP fusion

THE THREE TRANSFORMER FAMILIES, COMPLETE / 2

- Sessions 7–10 span this whole picture: attention and architecture (7), the encoder-only and encoder-decoder families in depth plus first cross-domain evidence (8), the decoder-only family and everything built on top of a pretrained LM – fine-tuning, prompting, reasoning, scaling, training cost (9, this session) – and the full non-NLP landscape (10).
- One architecture, three attention patterns, four sessions – and, per session 9, a training and reasoning story that is still actively being rewritten.

References

REFERENCES

- Radford, A., et al. (2018). Improving Language Understanding by Generative Pre-Training. OpenAI.
- Radford, A., et al. (2019). Language Models are Unsupervised Multitask Learners. OpenAI.
- Brown, T., et al. (2020). Language Models are Few-Shot Learners. https://arxiv.org/abs/2005.14165
- Schick, T., & Schütze, H. (2020). Exploiting Cloze Questions for Few-Shot Text Classification and Natural Language Inference. https://arxiv.org/abs/2009.07118
- Kwiatkowski, T., et al. (2019). Natural Questions: A Benchmark for Question Answering Research. https://aclanthology.org/Q19-1026/
- Wei, J., et al. (2021). Finetuned Language Models Are Zero-Shot Learners (FLAN). https://arxiv.org/abs/2109.01652
- Sanh, V., et al. (2021). Multitask Prompted Training Enables Zero-Shot Task Generalization (T0). https://arxiv.org/abs/2110.08207

REFERENCES / 2

- Wei, J., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. https://arxiv.org/abs/2201.11903
- Yao, S., et al. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. https://arxiv.org/abs/2305.10601
- Wei, J., et al. (2022). Emergent Abilities of Large Language Models. https://arxiv.org/abs/2206.07682
- Schaeffer, R., Miranda, B., & Koyejo, S. (2023). Are Emergent Abilities of Large Language Models a Mirage? https://arxiv.org/abs/2304.15004
- Quentin, A., et al. (2023). Transformer Math 101. EleutherAI Blog. https://blog.eleuther.ai/transformer-math/
- Kaplan, J., et al. (2020). Scaling Laws for Neural Language Models. https://arxiv.org/abs/2001.08361
- Hoffmann, J., et al. (2022). Training Compute-Optimal Large Language Models (Chinchilla). https://arxiv.org/abs/2203.15556

REFERENCES / 3

- Dao, T., et al. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. https://arxiv.org/abs/2205.14135
- DeepSeek-AI (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. https://arxiv.org/abs/2501.12948
- Nature (2025). DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. https://www.nature.com/articles/s41586-025-09422-z
- Reuters (2024). OpenAI and others seek new path to smarter AI as current methods hit limitations. https://www.reuters.com/technology/artificial-intelligence/openai-rivals-seek-new-path-smarter-ai-current-methods-hit-limitations-2024-

License and Attribution

LICENSE AND ATTRIBUTION

- Most of this session adapts material from the DL4NLP course (`lecture_dl4nlp`), chapters 7 (GPT), 9 (LLMs), and 11 (Training LLMs) – licensed CC BY 4.0, same as the rest of this course's source material (sessions 5–8, 10).
- All figures in this deck are copied directly from that source material's own `figure/` directories, under the same CC BY 4.0 license – unlike session 10, where external, differently-licensed source figures were deliberately left out.
- The taxonomy table, section framing, transition text, the emergent-abilities skepticism frame, and all callbacks to sessions 7/8/10 are freshly authored for this course, not copied from DL4NLP.
- Sourcing and every condensation decision are documented in this session's `order.txt`.